



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

TicketTailor

Architecture and Evaluation Report

CSSE6400 — Software Architecture
Semester 1, 2026

Authors

Jordan Lazo Cahua	s4713085
Hani Nursyamira	s4977217
Josephine Lim	s4730571
Irene Ge	s4985263
Daniel Cottrell	s4699165

June 7, 2026

Abstract

This report presents the architectural design, implementation, and evaluation of Ticket-Tailor, a web-based event management and RSVP platform designed to handle high-concurrency traffic spikes. To balance database consistency with low-latency responsiveness, a modular monolith backend was designed to separate core business domains into decoupled schemas. This is backed by a Redis caching layer for atomic RSVP counters and a PostGIS geospatial search engine. Additionally, resource-heavy background tasks (email and web-push notifications) are isolated using a transactional outbox pattern and an asynchronous, serverless worker. To validate the architecture, a test suite was implemented, ranging from unit tests and contract fuzzing to browser-driven end-to-end tests and load tests on AWS. The evaluation demonstrates that the architecture scales successfully under peak traffic and maintains strict data integrity during concurrent spikes, proving that the modular monolith and serverless notification engine design provides a robust, reliable, and production-ready foundation for the platform's requirements.

Contents

1	Project Scope Changes and Design Adjustments	3
1.1	Summary of Changes	3
1.2	Justification and Impact	3
2	Architecture Options	5
2.1	Option 1: Traditional Two-Tier Monolith	5
2.2	Option 2: Full Microservices	5
2.3	Option 3: Modular Monolith with Extracted Relay and Worker	6
2.4	Selected Option and Rationale	6
3	Architecture	7
3.1	System Context (C4 Level 1)	7
3.2	Containers (C4 Level 2)	7
3.3	Components: Inside the Modular Monolith (C4 Level 3)	9
3.4	Dominant Runtime Flow: Reliable Asynchronous Fan-out	11
3.5	Satisfying the Architecturally Significant Requirements	11
4	Trade-offs and Architectural Decisions	14
4.1	Consistency vs. Low-Latency Scalability (RSVP Counts)	14
4.2	Stateless Workers vs. Database Connection Pools	14
4.3	At-Least-Once Delivery vs. Message Duplication	15
4.4	Strict Module Boundaries vs. Development Friction	15
5	Architectural Critique	16
5.1	Critique 1: Scalability	16
5.2	Critique 2: Reliability – Transactional Outbox	16
5.3	Critique 3: Reliability – Redis & Read Replica	17
5.4	Critique 4: Interoperability – iCal Export	17
5.5	Critique 5: Notification Worker & Database-Free Constraints	18
5.6	Critique 6: Security and Privacy	18
6	Evaluation	19
6.1	Evaluation Approach	19
6.2	Functional Correctness	19
6.3	Scalability Evaluation	20
6.4	Reliability Evaluation	20
6.5	Interoperability Evaluation	20
6.6	Security and Privacy Evaluation	21
6.7	Limitations	21
7	Reflection	22
7.1	What Was Learned	22
7.2	What Would Be Done Differently	22
A	Supplementary Diagrams	24
A.1	Entity-Relationship Diagram	24
A.2	Sequence Diagram: RSVP Placement	25
A.3	Sequence Diagram: Authentication Flow	26
A.4	Sequence Diagram: Visibility Transition	27
A.5	Sequence Diagram: Browse Events	28
A.6	Sequence Diagram: Calendar Export	29

1 Project Scope Changes and Design Adjustments

During the development and testing cycle, several adjustments were made to the original project proposal. These changes were driven by the need to safeguard database resources, adapt to assessment environment constraints, simplify deployment in the learner lab, and ensure local testing reproducibility.

1.1 Summary of Changes

The following modifications were made to the proposed design:

- **Migration of Notification Worker to AWS Lambda (ADR-0002):** Originally planned as a continuously running ECS Fargate task, the notification worker was migrated to an AWS Lambda function triggered by SQS Event Source Mapping. This avoids paying for idle compute resources, scales compute down to zero when there are no notifications, and scales out dynamically to process fan-outs concurrently in response to SQS queue depth.
- **Folding the Scheduled Visibility Transition (ADR-0010):** Rather than deploying an independent scheduled Lambda function and EventBridge trigger rule to transition event visibility from club-only to public, this scheduled logic was folded directly into the Outbox Relay daemon. In every tick, the relay runs a sequential transactional query: it flips due club-only events to public and adds the `visibility_changed` rows to the outbox table, improving the scheduling resolution from minutes to seconds.
- **Relay-Side Recipient Resolution and Hydration (ADR-0016):** The task of resolving club followers and event attendees, as well as hydrating their contact details (emails and VAPID push keys), was shifted to the Outbox Relay. The notification worker was kept database-free and stateless, receiving pre-hydrated payloads from SQS. The relay queries the database (via service module boundaries) to resolve followers or RSVP attendees, fanning out one fully hydrated message per recipient to SQS. This protects the database connection pool from being exhausted if hundreds of Lambda instances spin up concurrently during a burst.
- **Frontend Framework Shift to Next.js Static Export (ADR-0018):** Originally proposed as a React 18 + Vite + TypeScript SPA, the frontend was migrated to Next.js with a static HTML export to utilise layouts and Tailwind configurations built in a frontend prototype. To maintain S3 + CloudFront static CDN hosting and the JSON-API contract, the Next.js bundle is built as static files. Dynamic segment routes (like `events/[id]`) were refactored into client-side query-parameter pages (like `events/detail?id=...`), and the TypeScript strict checking was deferred to hit the project deadline.
- **Email Provider Swap to Resend and Mailpit (ADR-0012):** The email transport was changed from AWS Simple Email Service (SES) to Resend (via REST API) for the deployed sandbox. Locally, Mailpit was integrated as a local SMTP capture sink in the Docker Compose environment. This ensures that load testing or manual runs do not consume commercial email quotas or send real emails to fake test addresses.

1.2 Justification and Impact

These design changes did not affect the core functional requirements of the Minimum Viable Product (MVP), but they significantly improved the system's quality attributes:

- **Reduction in Deployable Units (Operational Simplicity):** Folding the visibility cron into the Outbox Relay and using static HTML exports served from Amazon S3

reduced the number of active running containers in the cloud. The backend architecture was kept to exactly three deployable units (API, Outbox Relay, and Notification Lambda), simplifying the Terraform code and lowering runtime infrastructure overhead.

- **Database Connection Protection (Scalability and Reliability):** Moving recipient hydration to the Outbox Relay was critical to prevent connection exhaustion. In the original design, the Lambda worker would query PostgreSQL to retrieve recipient details. Under a burst of 1000 notifications, hundreds of concurrent Lambda instances would spin up, instantly saturating the relational database's connection pool. By keeping the worker database-free, it runs as pure stateless compute, protecting database availability for client-facing API requests.
- **Adaptation to Academic Cloud Constraints (Security and Privacy):** UQ assessment policies restricted the use of AWS SES. Swapping to Resend allowed the project to execute real email alerts under sandbox limits without AWS credit card verification or sandbox request limits. Additionally, Mailpit catches all SMTP traffic locally, preventing real email blasts during heavy automated load tests.

The remainder of this page is intentionally left blank.

2 Architecture Options

This section compares the main architectural options considered for TicketTailor and explains why the team selected a modular monolith with extracted event-driven components. The comparison is based on the project’s architecturally significant requirements (ASRs): scalability for RSVP and notification spikes, reliability for RSVP counts and event state changes, interoperability with external calendar tools, and security/privacy for user and event data.

2.1 Option 1: Traditional Two-Tier Monolith

The simplest option was a traditional two-tier architecture: a single web application connected directly to a relational database. This option would have been the fastest to build and easiest to deploy. It would also make local development and functional testing straightforward because most behaviour would live in one application and one database.

However, this option was not suitable for TicketTailor’s ASRs. The system needs to handle bursty RSVP traffic, public event browsing, scheduled visibility transitions, and notification fan-out when events change. In a basic two-tier design, heavy work such as notification delivery or calendar/feed generation would share the same request path as ordinary API requests. This increases the risk that expensive background work slows down user-facing operations. It also provides a weak scalability story because there is no clear asynchronous boundary for notification bursts and no credible path for extracting hot modules later.

For these reasons, the two-tier option was rejected. It was simple, but it did not provide enough architectural depth to support the scalability, reliability, and extensibility requirements of the project.

2.2 Option 2: Full Microservices

A second option was a full microservices architecture. In this design, major domains such as authentication, users, organisers, events, RSVP, calendar, notifications, and map/search functionality would be split into independently deployed services, likely behind an API gateway and connected through service-to-service communication and message queues.

The main advantage of this option is strong separation and independent scalability. For example, an RSVP service, calendar service, and notification service could each scale separately based on their own workloads. This would also provide clear service ownership and a strong long-term expansion path.

The drawback is that this option adds significant operational and testing complexity. A full microservices design would require multiple deployments, inter-service contracts, distributed tracing, service discovery, and more complicated failure handling. It would also make data consistency harder, especially for workflows such as RSVP placement, attendee count updates, event updates, and notification generation. These workflows must remain reliable under load, and coordinating them across multiple services would increase implementation risk within the semester timeline.

Because the course project rewards a working implementation, test quality, architecture description, and architecture evaluation, the team decided that full microservices would create more complexity than value for the MVP. Although it is more expandable in theory, it would reduce the team’s ability to thoroughly implement and evaluate the core architecture.

2.3 Option 3: Modular Monolith with Extracted Relay and Worker

The selected option was a modular monolith for the main API, combined with extracted event-driven components for asynchronous work. The core API remains one deployable backend, but it is organised into clear domain modules such as authentication, users, organisers, events, RSVP, calendar, and shared infrastructure. These modules use service and repository layers, separate database schemas, and import-boundary checks to avoid becoming a tangled monolith.

This option keeps the main system manageable while still providing architectural separation. It avoids the deployment and testing overhead of full microservices, but it is more sophisticated than a basic two-tier application. The extracted outbox relay and notification worker create an explicit asynchronous boundary for work that should not block the API request path. This is especially important for notification fan-out and visibility transitions.

For reliability, the architecture uses a transactional outbox pattern. Business changes such as RSVP actions or event updates can be committed in the same transaction as an outbox record. The relay then publishes those events asynchronously for downstream notification processing. This avoids the risk of an API request succeeding while the associated notification event is lost.

For scalability, RSVP counts use a Redis-backed live counter while PostgreSQL remains the durable source of truth for RSVP rows. This reduces contention on a single relational counter during bursts, while reconciliation can correct counter drift from the database record. Read-heavy and search-heavy operations are also supported by read-oriented infrastructure, while the extracted worker can scale separately for notification workloads.

For interoperability, the calendar module exposes event data using iCalendar-compatible output rather than a provider-specific integration. This allows the MVP to support external calendar tools while keeping OAuth-style two-way calendar integration as a stretch or future enhancement.

2.4 Selected Option and Rationale

The team selected the modular monolith with extracted relay and worker because it best balances the MVP constraints with the required quality attributes. It gives the project a clear architecture story: keep closely related business domains in one API for consistency and testability, but extract the bursty and asynchronous notification path where the ASRs justify it.

Compared with a simple two-tier monolith, the selected architecture better supports scalability and reliability because notification fan-out, visibility transitions, Redis counters, and outbox processing are not treated as ordinary synchronous request work. Compared with full microservices, it avoids unnecessary distributed complexity while still preserving a credible future extraction path through enforced module boundaries.

This design is also practical for the learner-lab deployment environment. The API, relay, worker, relational database, Redis, and messaging components can be deployed and evaluated as a realistic cloud architecture without requiring the team to manage a large number of independently deployed microservices. Overall, the selected architecture provides enough sophistication to satisfy the ASRs while remaining feasible to implement, test, and evaluate within the project timeline.

The remainder of this page is intentionally left blank.

3 Architecture

The option selected in Section 2 is realised as a modular-monolith API supported by two extracted event-driven processes, a small set of specialised datastores, and a single-page web client. This section describes that realisation using the C4 model (system context, containers, and components), then traces the dominant runtime flow and sets out how the structure satisfies each architecturally significant requirement (ASR). The figures are generated from the authoritative diagram sources under `model/artefacts/`; the full set, including the entity-relationship model and the remaining sequence diagrams, lives there.

3.1 System Context (C4 Level 1)

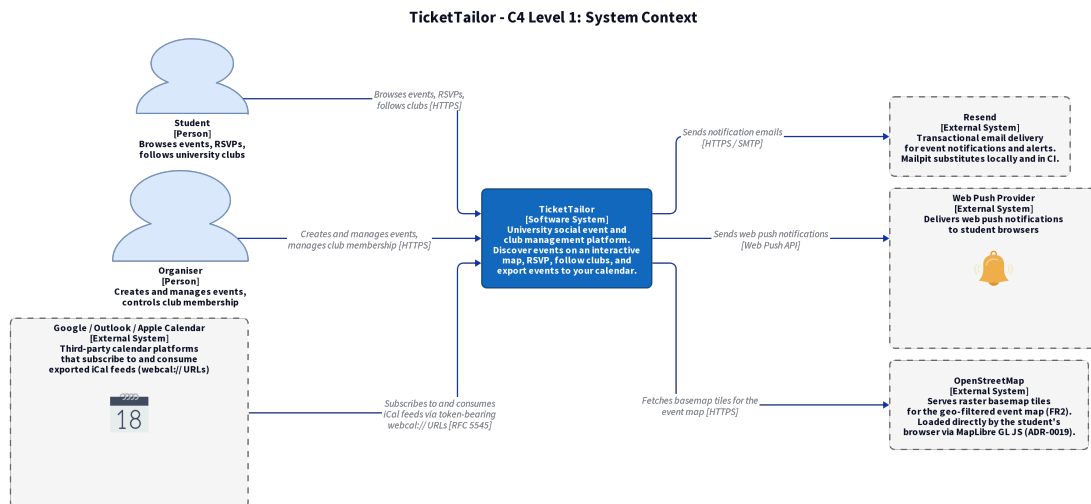


Figure 1: C4 Level 1: system context. Students and committee members use one web client against a single public API, which integrates with four external services.

Two human roles interact with TicketTailor. Students browse events on a geo-filtered map, RSVP, follow clubs, and receive notifications; committee members additionally create and manage events and configure pricing and visibility. Both use the same single-page web client, which talks to one public HTTP API.

The system integrates with four external services rather than re-implementing their concerns. Event details are exported as standards-compliant iCalendar feeds that any calendar client can consume (ADR-0007), so interoperability is achieved through an open format rather than per-vendor integrations. Real-time notifications are delivered through the browser's native Web Push service using VAPID (ADR-0011) and through an email provider, Resend (ADR-0012). Map tiles are loaded client-side directly from OpenStreetMap (ADR-0019), keeping tile traffic off the API entirely. Treating these as external systems keeps the core platform small and the trust boundary clear.

3.2 Containers (C4 Level 2)

The platform comprises three independently deployable backend units, three datastore concerns, and one frontend, shown in Figure 2.

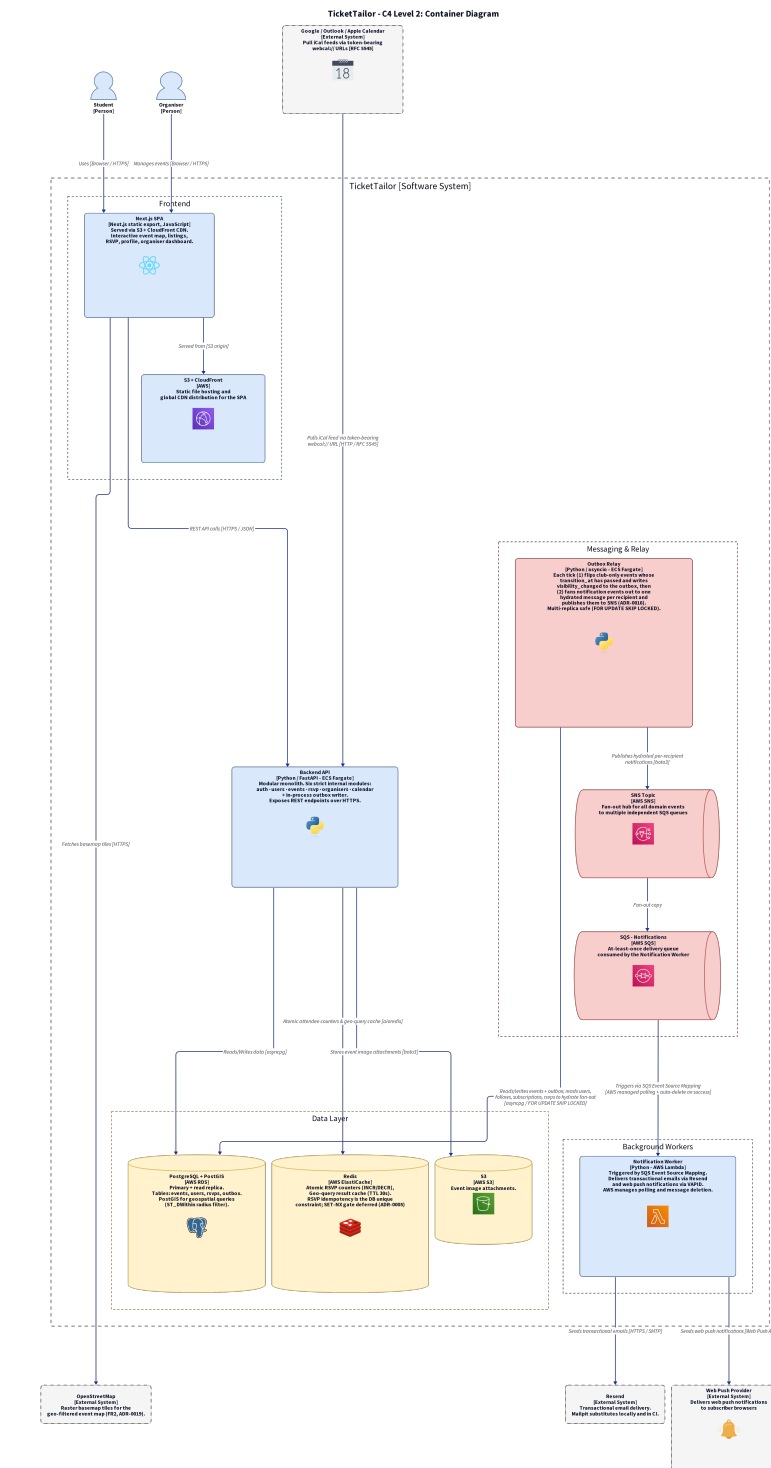


Figure 2: C4 Level 2: containers. Three deployable backend units, three datastore concerns, and one static frontend.

The API is the modular monolith: a FastAPI application (ADR-0003) that handles all synchronous request and response traffic behind an application load balancer, and the only unit that accepts external requests. The outbox relay and the notification worker are extracted from the API precisely because their work is asynchronous and bursty and must not share the API's request path. The relay polls the transactional outbox, publishes domain events to the message bus, and additionally runs the scheduled FR5 visibility transition, the time-offset flip from club-only to public (ADR-0010). The worker consumes notification messages and performs the actual push and email delivery.

Three datastores back these units. A PostgreSQL primary with the PostGIS extension is the durable source of truth for relational data and geo-queries, with at least one read replica so that read-heavy map traffic does not contend with writes (ADR-0008). A Redis instance holds the live RSVP counter and short-lived caches, serving atomic `INCR` and `DECR` operations that would otherwise bottleneck on a single relational row (ADR-0005). An SNS topic fanned out to an SQS queue is the asynchronous boundary between the relay as publisher and the worker as consumer, with a dead-letter queue for messages that exhaust their retries (ADR-0006).

The frontend is a Next.js application compiled to a static bundle (ADR-0018); it holds no server-side logic and reaches the API over the same JSON contract as any other client. On the deployment target, the API and relay run as ECS Fargate services with independent auto-scaling, the worker runs as an AWS Lambda triggered by an SQS event-source mapping, and the datastores use managed equivalents (RDS, ElastiCache, SNS and SQS). The AWS Academy Learner Lab profile the project is actually deployed under is recorded in ADR-0017, with the static frontend served over HTTP from an S3 website on the lab (ADR-0020); neither substitution changes the container boundaries.

3.3 Components: Inside the Modular Monolith (C4 Level 3)

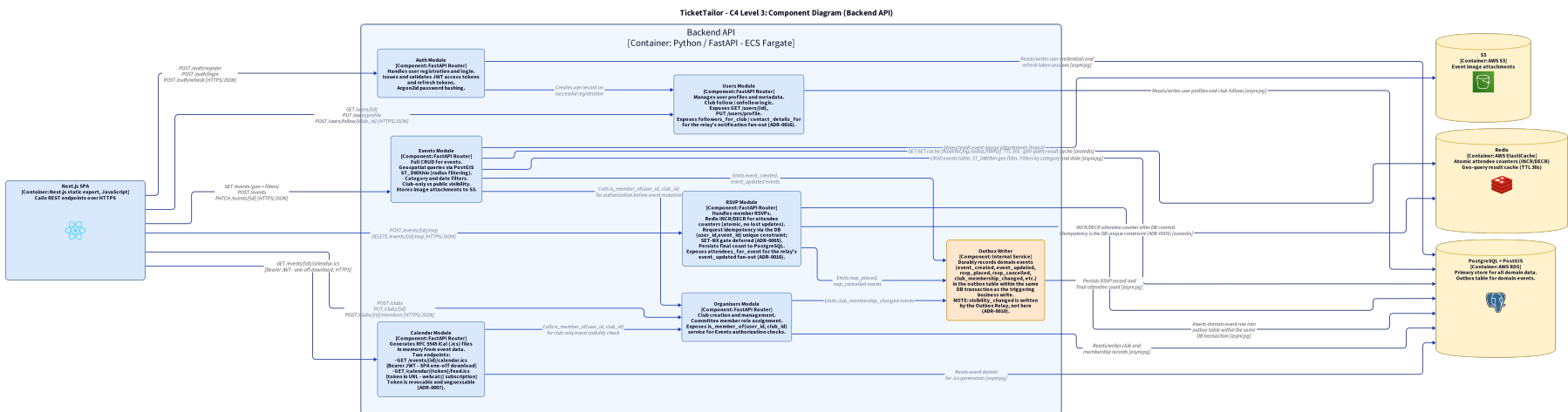


Figure 3: C4 Level 3: components of the API container. Bounded domain modules over a shared infrastructure layer, each internally layered into router, service, and repository.

Figure 3 opens the API container. It is partitioned into bounded domain modules for authentication, users, organisers, events, RSVP, and calendar, over a shared infrastructure layer. Each module is internally layered into a router for the HTTP surface, a service for business logic and authorisation, and a repository for persistence, which keeps authorisation logic testable in isolation rather than scattered through request handlers.

The decisive property is that these boundaries are enforced, not merely documented. Modules communicate only through one another's service interfaces; no module imports another's repositories or ORM models. The contract is checked mechanically in CI by import-linter (ADR-0002), so a boundary breach fails the build. Each module additionally owns its own database schema and migration history (ADR-0013), and cross-module needs are met through narrow, explicit seams: identity is supplied to every router by a single shared authentication dependency (ADR-0014), and cross-module invariants, such as confirming a club exists before a follow, are checked through registered shared callbacks rather than direct calls into another module's internals (ADR-0015). The result is a single deployable that is nonetheless decomposed cleanly enough that any module could later be extracted to its own service with a bounded, known change; that per-module extraction path is the project's answer to the expandable-to-a-larger-system criterion.

3.4 Dominant Runtime Flow: Reliable Asynchronous Fan-out

The architecture's most important dynamic behaviour is how a change inside the API reliably becomes a notification without coupling the two, traced in Figure 4. When a business write occurs, such as an RSVP, an organiser edit, or the visibility flip, the API commits the domain change and an outbox row in the same database transaction. Either both persist or neither does, so a request can never succeed while its notification event is silently lost. The relay then polls unpublished outbox rows, an index-backed scan that is proportional to the pending set rather than the whole table, and publishes each as a message to SNS. SNS fans the message out to the SQS queue, which triggers the worker. Because the relay hydrates each message with its recipient set before publishing (ADR-0016), the followers of the club for an FR5 transition or the RSVP'd attendees for an organiser edit, the worker delivers per recipient without reaching back into the database. Delivery is at-least-once: a transient failure is reported back to SQS so only the failed message is redelivered, and a message that exhausts its retries lands in the dead-letter queue rather than being dropped.

This single pattern carries much of the scalability and reliability story, set out in the next subsection.

3.5 Satisfying the Architecturally Significant Requirements

For scalability (QA1), the spike scenario, in which a popular event flips to public and notifies every follower while map traffic continues, is absorbed at four points. The notification worker is a separate process, so it scales as Lambda concurrency independently of the API; the SNS-to-SQS boundary decouples the rate of publishing from the rate of delivery, letting a backlog drain without back-pressuring the API; the read replica keeps the geo-query path off the write primary; and the Redis counter removes RSVP-count contention from a single database row. The headline load test delivered a fan-out to 300 followers in full with notification-ack p95 around 5 s, roughly six times under the 30 s target, and argues the 1,000-follower target by extrapolation, since ack latency stayed flat as the follower count grew (Section 6).

For reliability (QA3), cross-replica correctness of attendee data is guaranteed by a combination of tactics, not replication alone. RSVP writes are idempotent through a unique constraint on (`user_id`, `event_id`), so a retried or duplicated toggle cannot double-count. The transactional outbox makes cross-process events durable, as described above. A reconciliation job periodically

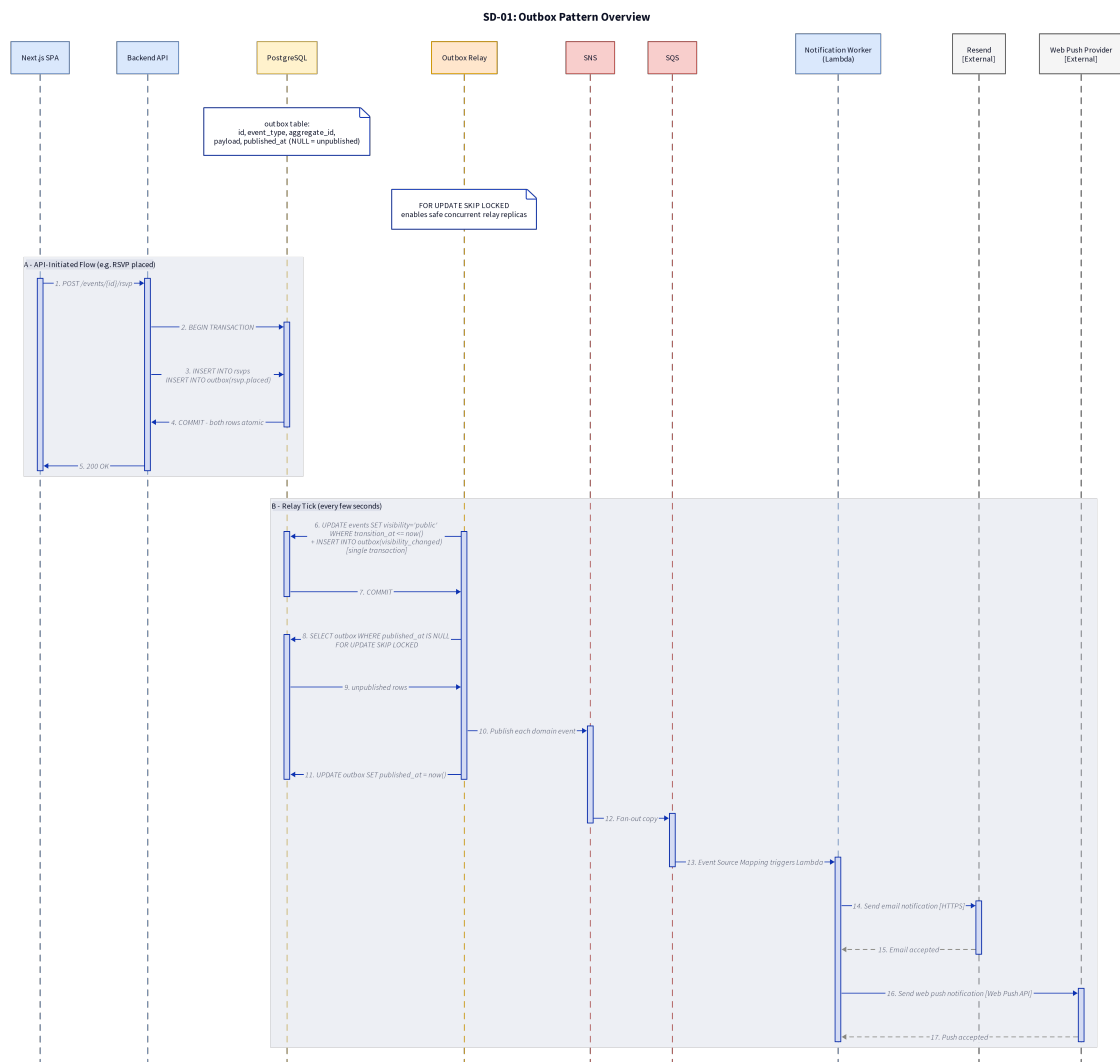


Figure 4: Outbox overview: a business write and its outbox row commit in one transaction; the relay publishes to SNS, which fans out to SQS and triggers the worker.

compares the live Redis counter against the persisted RSVP rows and corrects any drift, and the at-least-once SQS delivery with a dead-letter queue ensures no notification is lost to a worker failure mid-fan-out. These are exercised directly by the reliability tests, including a primary-failover-mid-burst case (Section 6).

For interoperability (QA2), the system emits RFC 5545 iCalendar feeds (ADR-0007) rather than integrating with each calendar vendor. A user saves an event once and the feed is consumed by Google, Outlook, or Apple Calendar; conformance is checked against an RFC 5545 validator and a manual round-trip. Choosing an open standard over vendor APIs keeps the surface small and avoids per-provider OAuth coupling, which is deferred as a stretch goal.

For security and privacy (QA4), authentication uses short-lived access tokens with rotating refresh tokens, and passwords are hashed with the memory-hard argon2id function (ADR-0009). Authorisation is role-based and lives in the service layer, so a student cannot mutate another club's events; ownership is checked through the organisers module rather than by trusting the request. Every request body is validated against a strict schema that rejects unknown fields, and the API's cross-origin policy is restricted to the deployed frontend origin. Transport is HTTPS in the idealised deployment; the Learner Lab's HTTP-only constraint and its mitigations are discussed in the critique (Section 5).

Taken together, the structure keeps the common case simple, one cohesive API over one relational store, while isolating exactly the bursty, failure-prone notification path to which the quality attributes are most sensitive.

The remainder of this page is intentionally left blank.

4 Trade-offs and Architectural Decisions

In designing the TicketTailor platform, several competing architectural forces were balanced. Key decisions involved trade-offs between system performance, state consistency, operational complexity, and data privacy to satisfy the project’s architecturally significant requirements (ASRs).

4.1 Consistency vs. Low-Latency Scalability (RSVP Counts)

To handle concurrent RSVP traffic spikes without saturating the primary relational database, read-path consistency was traded for low-latency scalability. Managing seat availability and attendee counts directly in PostgreSQL through transactional row-level updates or active aggregates (`COUNT(*)`) introduces severe lock contention and transaction failures under heavy write bursts. To prevent this, the live counter path is decoupled from PostgreSQL by utilizing a fast in-memory Redis cache line (`rsvp:count:{event_id}`). Read operations for map browsing and event lists query the Redis counter directly.

This design introduces the risk of cache-to-database read drift. The Redis counter can temporarily diverge from the authoritative database rows if Redis experiences a crash or key eviction. However, the counter is only updated after a successful PostgreSQL commit, so a failed transaction never leaves a phantom counter. This drift was accepted because user-facing read updates do not require immediate transaction-level consistency. To safeguard correctness, the RSVP service executes a strict order of operations: the SQL row is inserted or deleted and durably committed to PostgreSQL before the write-through increment or decrement is sent to Redis. Seeding a cold key also runs before database writes to prevent double-counting or under-counting base values. Furthermore, a periodic background reconciliation task recalculates persisted RSVP rows in PostgreSQL and overwrites drifted Redis values. If Redis becomes entirely unreachable, the client connection wrapper raises connection timeouts quickly, allowing the API to degrade gracefully by falling back to SQL database queries instead of blocking the main ASGI event loop.

4.2 Stateless Workers vs. Database Connection Pools

To scale the notification engine under heavy bursts, the team faced a trade-off between database resource safety and notification boundary complexity. In a naive event-driven design, the notification worker (AWS Lambda) would receive thin event payloads and query the database to resolve recipient contact details and push subscriptions. However, fanning out updates concurrently to hundreds or thousands of club followers would spin up many parallel Lambda instances. This concurrent spike would quickly exhaust the PostgreSQL primary database connection pool, starving the core API and crashing the database.

To prevent connection starvation, the notification worker was designed to be completely database-free and stateless. Recipient resolution and message hydration are shifted upstream to the outbox relay running on ECS Fargate. The relay queries the database through module service interfaces to obtain emails, display names, and VAPID push keys, fanning out one fully pre-hydrated message per recipient to SNS, which fans out to the subscribed SQS queues. The trade-off is that fanning out hydrated messages widens the relay’s dependencies and places personally identifiable information (PII) on the message bus. This security risk is mitigated by enabling server-side encryption with AWS managed keys (SSE-KMS) on the SNS topics and SQS queues. The database-free worker can scale rapidly without VPC or RDS connection bottlenecks, but because it has no database access, it cannot clean up expired push keys directly when push services return subscription errors. Instead, it logs these occurrences, leaving active database pruning as a potential improvement to be managed via secure API callbacks in a future version.

4.3 At-Least-Once Delivery vs. Message Duplication

To guarantee that no critical notification is permanently lost, reliability was prioritized over avoiding duplicate message deliveries. In a highly concurrent environment, ensuring exactly-once delivery across distributed processes (monolith API, outbox relay, SQS, and Lambda worker) is functionally impractical without heavy coordination overhead that degrades performance. The architecture uses a transactional outbox pattern combined with SQS retries to enforce an at-least-once delivery boundary.

The trade-off is that both the outbox relay and the SQS queues can emit duplicate messages under ordinary failure conditions. The relay uses an all-or-nothing publish strategy: all messages for a single outbox row must be published successfully before the row is marked as published, so a partial failure leaves the entire row unpublished for full retry on the next tick. Similarly, if the Lambda worker encounters a transient error (such as an email provider timeout) during a concurrent delivery run, it raises an exception and reports failures to SQS. SQS then redelivers the message, which may result in duplicate emails or push notifications on already-successful delivery channels. This duplicate overhead was accepted as a necessary compromise to ensure zero messages are lost. To mitigate the impact on users, the outbox relay generates deterministic message IDs based on the outbox row ID and recipient details, allowing downstream clients or worker handlers to identify and deduplicate incoming messages.

4.4 Strict Module Boundaries vs. Development Friction

To support future extensibility, the ease of ad-hoc database queries was traded for logical isolation and modular independence. In a traditional two-tier monolith, developers routinely write SQL queries that join tables across arbitrary modules or import classes globally, creating tightly coupled spaghetti code. To prevent this, strict schema-per-module separation is enforced, and physical database foreign keys across module schemas are prohibited. Module boundaries are also checked during CI runs using Import Linter.

This strict logical isolation introduces significant development friction. Common tasks, such as verifying club membership before allowing an event RSVP or retrieving event titles for organizer notifications, cannot use direct database joins or repository imports. Instead, modules must coordinate through narrow, service-level interfaces, registered shared callbacks, or multi-query aggregates. This structure increases the size of the codebase and the complexity of simple operations. However, the trade-off is justified because it prevents architectural degradation. The enforced boundaries ensure that the system maintains high cohesion and loose coupling, providing a clean extraction path where any single module can be migrated to a standalone microservice in a future version without refactoring the remaining codebase.

The remainder of this page is intentionally left blank.

5 Architectural Critique

The TicketTailor architecture has been evaluated against its core Architecturally Significant Requirements (ASRs) to see how well the system supports high concurrent loads, state reliability, and external integrations. By looking at the benefits, trade-offs, and mitigations of the design choices, the overall strength of the modular monolith and serverless worker model can be assessed.

5.1 Critique 1: Scalability

The architecture is designed to handle sudden concurrent RSVP spikes and large notification bursts. To achieve this, Redis atomic counters manage seat availability in memory [1]. By decrementing the ticket count in Redis before writing to the database, the system prevents lock contention and transaction failures on the primary PostgreSQL database under heavy load. For notifications, the background worker uses SQS queues and AWS Lambda, which scales out automatically as the queue grows, ensuring the main API does not slow down. Also, because the platform uses a modular monolith on ECS Fargate, modules communicate locally in-process. This avoids the network latency and serialization overhead associated with microservices.

However, sharing the same runtime brings some trade-offs. Because all modules run inside the same ECS Fargate task, a resource-heavy task in one module—like generating a calendar or processing files—can hog CPU or memory and slow down critical paths like RSVPs. Also, since all modules share the same database instance, heavy geospatial searches on the map can consume connection pools and impact write transactions. Lastly, scaling is coarse-grained; the entire API container must scale even if only one module is bottlenecked.

To mitigate this, Import Linter enforces strict module boundaries. The system's `.importlinter` configuration defines a strict layered hierarchy where the topmost layer, `tickettailor.calendar`, can access lower layers like `tickettailor.rsvp`, `tickettailor.events`, `tickettailor.organisers`, `tickettailor.auth`, and `tickettailor.users`, while the lowest layer, `tickettailor.shared`, is prohibited from importing anything above it. This keeps the code decoupled, meaning the RSVP or search module can be easily extracted and turned into a separate microservice should traffic scale up in a hypothetical future version.

5.2 Critique 2: Reliability – Transactional Outbox

It is critical that notifications are not lost and that the system state stays consistent under high concurrency. This is solved by implementing the transactional outbox pattern [2]. By saving the business record (like a new RSVP) and the notification event inside the same database transaction, the system guarantees that notification requests are never lost even if the application crashes mid-request. Once saved, the outbox relay service processes the events asynchronously.

The outbox relay service runs a dual-pass work loop in each tick. In the first phase, it performs a scheduled visibility transition, updating events from `club_only` to `public` when they are due and adding `visibility_changed` events to the outbox table. In the second phase, it pulls pending outbox rows, resolves the recipients, and hydrates the messages with recipient details (email, display name, latest VAPID push keys) using service interfaces like `followers_for_club` and `attendees_for_event`. Both phases utilize `SELECT ... FOR UPDATE SKIP LOCKED` queries. This database-level row locking allows multiple concurrent relay replicas to run safely without double-processing events or causing database deadlocks.

The main trade-off is the extra load on the database. Because the outbox relay constantly polls the database to find new events, it puts a steady read and write load on the primary

PostgreSQL database [2]. It also adds some architectural complexity since a background polling daemon must be managed.

To minimize this database load, monitoring tracks the outbox table size and relay latency. This allows tuning of the polling intervals and cleanups of processed outbox rows, keeping the database running smoothly.

5.3 Critique 3: Reliability – Redis & Read Replica

Attendee counts and event data must also remain reliable under load. Using Redis atomic decrements (DECR) works as a fast buffer that stops concurrent write bursts from hitting the relational database directly [1]. The design also offloads read-heavy operations, like geospatial map searches, to a PostgreSQL read replica to protect the primary database's connection pool. On top of that, API-level idempotency keys drop duplicate RSVP requests and avoid database corruption.

The implementation details of the RSVP counter path show strict execution order to prevent phantom counts. During a write transaction, the RSVP service first seeds the Redis counter from the database on a cold key before inserting the new row. This ensures the write-through increment lands on the correct base. The new RSVP row and outbox event are then committed to PostgreSQL before incrementing the Redis counter. If the database commit fails, the Redis write never runs, avoiding phantom counts. The Redis client is also configured with socket connection timeouts so that if Redis becomes unreachable (for example, during cache failover), the client raises an error quickly. The API then degrades gracefully by falling back to the database count instead of hanging the event loop.

To manage the database connection pool under heavy concurrent traffic, the SQLAlchemy client bounds maximum connections using pool sizes and overflow configurations. It also uses pre-ping checks to discard stale connections. Execution timeouts on both the client and server side ensure that slow queries cannot pin pooled connections indefinitely under load.

Additionally, while cache drift can occur due to restarts or evictions, it is corrected by a periodic reconciliation task. This task uses non-blocking Redis key scans (SCAN) instead of blocking lookups to protect concurrent performance while overwriting drifted values with the PostgreSQL truth.

5.4 Critique 4: Interoperability – iCal Export

For interoperability, users must be able to sync events with external calendars like Google Calendar, Outlook, and Apple Calendar. This is achieved by generating calendar feeds that follow the RFC 5545 standard, which guarantees compatibility without custom code for each provider [3]. These feeds are generated dynamically in memory, saving the system from storing static files on disk. Users can subscribe to updates using the `webcal://` protocol with tokenized URLs, allowing external calendar apps to pull updates automatically.

The implementation maps events into standard properties like calendar scale, method, starts/ends times, and geo coordinates. To secure these feeds, tokens are generated using cryptographically secure strings. The API supports token rotation, allowing users to revoke old tokens and generate new ones, immediately invalidating leaked URLs. The feed generation also enforces strict authorization: if a user is subscribed to a club-only event but is subsequently removed from that club, the feed query checks their membership status and dynamically filters out the restricted events. These read-only feed queries are executed against the read replica database session to prevent heavy join operations from impacting primary write transactions.

The trade-off is that this integration is strictly read-only, so users cannot edit events or RSVP

directly from their calendar apps. Also, the system cannot control when the external calendar updates; they pull data on their own schedules, which can cause delays.

5.5 Critique 5: Notification Worker & Database-Free Constraints

The notification worker is designed as a stateless, database-free AWS Lambda function. By avoiding database connections and VPC network configuration, the Lambda worker can scale rapidly without connection-pool exhaustion, resolving a common bottleneck when fanning out hundreds of concurrent notifications. However, this statelessness imposes constraints on handling delivery failures.

If a push notification endpoint returns a `410 Gone` error (indicating the subscription has expired), the database-free worker cannot delete the stale row directly. Instead, it logs the expiration, leaving active database pruning as a potential improvement to be considered for a future version, such as through a secure API callback or SQS event.

For delivery failures, the worker runs email and push tasks concurrently. If a transient error occurs (like a provider network timeout), the worker raises an exception. The Lambda handler then reports the failed record to SQS via `batchItemFailures`. SQS handles retrying the message, ensuring at-least-once delivery. A trade-off is that retries can lead to duplicate notifications on successful channels (e.g. an email is sent twice because a push notification failed on the first try), which is accepted to prevent permanent message loss.

5.6 Critique 6: Security and Privacy

In the idealised deployment, all external communication is encrypted via HTTPS on both the public API and the CDN-served frontend. However, due to AWS Academy Learner Lab permission restrictions preventing CloudFront provisioning, the system is deployed using a public S3 static website hosting the frontend and a public ALB serving the API, both operating over unencrypted HTTP (ADR-0020).

The trade-off of this temporary deployment model is that user credentials, authentication tokens, and sensitive event details are transmitted in clear text, exposing the system to eavesdropping and man-in-the-middle attacks. Additionally, the S3 bucket must remain publicly readable. While acceptable for a sandbox environment, this security gap is mitigated in the idealised architecture by setting `enable_frontend = true` in Terraform, which provisions a CloudFront distribution with HTTPS, and configuring an ACM SSL certificate with an HTTPS listener on the ALB.

The remainder of this page is intentionally left blank.

6 Evaluation

This section evaluates how well the implemented TicketTailor architecture supports the project's key functional requirements and architecturally significant requirements (ASRs). The evaluation combines automated test suites, deployed AWS load testing, calendar interoperability checks, browser-driven end-to-end tests, and security-oriented static analysis. The main ASRs evaluated are scalability, reliability, interoperability, and security/privacy.

6.1 Evaluation Approach

The system was evaluated at multiple levels. Automated backend tests were used to verify the correctness of authentication, users, organisers, events, RSVP, calendar export, validation, and health-check behaviour. Separate relay and worker tests were used to validate the asynchronous notification pipeline. Browser-driven Playwright end-to-end tests were included for the map UI and the AR3 happy path, and the traceability matrix records these tests as passing. Load tests were used to evaluate the deployed architecture under more realistic traffic conditions.

The Python components were evaluated with formatting, linting, type checking, import-boundary checks, Bandit static security analysis, and automated pytest suites. These checks passed locally for the API, relay, and worker components, with the API test suite passing 119 tests and 32 Schemathesis subtests. The API, relay, worker, web, and security workflows also passed on the final main branch, providing CI evidence for backend checks, asynchronous components, front-end build quality, and security scanning. The supporting test code is available in the project repository. API tests are located in the API module test folders, while relay tests, worker tests, browser-driven end-to-end tests, and load-test scenarios are stored in the repository's relay, worker, e2e, and load-test directories. The main recorded test reports are stored under `docs/test-reports`. Local API tests can be run from the `api` directory using `uv run pytest -vv`, while project quality checks are run through the configured pre-commit hooks.

The main load evaluation was run against the deployed AWS stack in `us-east-1`. The deployed environment used an ECS Fargate API behind an Application Load Balancer, autoscaling between 2 and 10 tasks, an RDS PostgreSQL primary with read replica, ElastiCache Redis, and an SNS-SQS-Lambda notification worker pipeline. Because the k6 harness was run from Australia against `us-east-1`, client-observed latency was affected by trans-Pacific network latency. For this reason, the report uses server-side API latency logs and worker CloudWatch acknowledgement timestamps as the meaningful performance measures.

6.2 Functional Correctness

The automated API tests cover the main MVP workflows. Authentication tests verify registration, login, token refresh, logout, and protected profile access. User and organiser tests verify profile management, club creation, membership management, following clubs, and role-based permissions. Event tests cover creation, retrieval, listing, updating, deletion, visibility behaviour, public browsing, geospatial search, cursor pagination, input validation, and authorisation. RSVP tests cover placement, cancellation, privacy checks for club-only events, Redis-backed counter behaviour, failover behaviour, and reliability of the outbox path. Calendar tests verify token flow and iCalendar output.

These tests show that the implemented backend supports the core functional requirements of the MVP. The Schemathesis tests also provide contract-style evidence that malformed or unexpected API inputs do not cause server errors. Together, the tests show that major features are not implemented as isolated endpoints only, but are integrated across modules through the service and repository layers.

6.3 Scalability Evaluation

Scalability was evaluated using deployed load-test scenarios for map traffic, RSVP traffic, and visibility-transition fan-out. The `map_traffic` scenario generated approximately 48 requests per second for eight minutes, producing 24,052 requests with 0 errors. During this run, the API autoscaled from a cold 2-task baseline up to 9 running tasks. Server-side geo-query p99 latency initially spiked while the 2-task baseline was saturated, but recovered as new tasks came online. Once the service settled, p99 latency reached approximately 136–202 ms, satisfying the 500 ms p99 target for the geo-search path.

The `sustained_rsvp` scenario produced 12405 RSVP toggles with 0 errors. Server-side latency was strong at the median and p95 levels, with p50 at 17 ms and p95 at 51 ms. However, the p99 result was 410 ms, which exceeded the stricter 200 ms write-path SLO. This means RSVP correctness and availability under load were achieved, but the tail latency of the primary database write path remains a limitation. The result is still useful architecturally because it shows that the Redis counter and database-backed RSVP path maintained correctness under load, while also identifying where future optimisation should focus.

The `visibility_spike` scenario tested the system’s ability to handle a club-only event becoming public and notifying followers. With 300 followers, the system delivered 300 out of 300 notifications, with acknowledgement p95 of 5.1 seconds and p99 of 5.3 seconds. The dead-letter queue remained empty. This satisfies the fan-out SLO and supports the claim that asynchronous notification processing can absorb bursty work without blocking the main API.

6.4 Reliability Evaluation

Reliability is supported by transactional writes, the transactional outbox pattern, Redis-backed counters, and asynchronous delivery with retry support. Automated tests verify that RSVP actions and event changes create the required outbox records. The relay and worker tests further validate that outbox events can be processed and delivered through the notification pipeline.

The deployed load test also provides reliability evidence. Across the sustained map and RSVP scenarios, the report recorded a 0% error rate. The fan-out test delivered all notifications and left the dead-letter queue empty. This supports the architecture’s reliability claim: user-facing API operations can complete without directly depending on notification delivery, while notification work remains durable and recoverable through the outbox and queue-based pipeline.

The evaluation also confirmed several deployment fixes under real AWS conditions. The worker handled an empty Resend key by falling back to logging, worker acknowledgement logs reached CloudWatch, request-count autoscaling operated in both directions, and the SNS–SQS–Lambda fan-out path delivered without loss.

6.5 Interoperability Evaluation

Interoperability was evaluated through the calendar export functionality. The calendar module generates iCalendar-compatible output so that events can be imported into external calendar platforms. Automated iCalendar conformance tests verify that generated calendar data follows expected formatting rules, while the manual calendar round-trip test confirmed that a generated event could be imported into Google Calendar with the expected title and time.

This provides evidence that the architecture supports interoperability through a standard format rather than a provider-specific integration. However, the evaluation is strongest for the tested Google Calendar path and automated RFC 5545 conformance. Broader manual testing across Outlook and Apple Calendar would provide stronger evidence for cross-platform behaviour.

6.6 Security and Privacy Evaluation

Security and privacy were evaluated through authentication tests, role-based authorisation tests, input validation tests, static analysis, and project security workflows. Authentication tests verify that protected endpoints reject unauthenticated users and invalid tokens. Organiser and event tests verify that users without the correct club role cannot perform restricted actions. Validation tests check that invalid or unsafe text input is rejected before it reaches sensitive parts of the system.

The architecture also includes privacy-specific behaviour for public event browsing. Public event coordinates are rounded before being returned, reducing the precision of location data while still allowing map-based discovery. Club-only events are filtered so that users who are not members of the relevant club cannot access restricted event information. These behaviours support the project's privacy and security goals in addition to the broader authentication and authorisation checks.

6.7 Limitations

The evaluation demonstrates that the architecture supports the MVP and its most important quality attributes, but there are still limitations. First, the load harness was run from Australia against a `us-east-1` deployment, so client-observed latency was affected by network distance. The server-side metrics are therefore the correct measure for evaluating the system itself, but an in-region load harness would provide cleaner end-to-end client latency data.

Second, the RSVP write-path p99 latency reached 410 ms under sustained load, exceeding the 200 ms SLO. This suggests that the primary database write path is the main bottleneck for the RSVP workflow. Potential improvements include increasing the primary database size, tuning API connection pools, or revisiting the strictness of the write-path SLO.

Third, organiser edit notification fan-out was not separately timed. The architecture uses the same relay, queue, and worker pipeline as the validated visibility fan-out scenario, but a separate organiser-edit load test would provide stronger evidence for that specific functional requirement.

Overall, the evaluation shows that the selected modular monolith with selective event-driven components is suitable for the MVP. The system scales for read-heavy map traffic, reliably delivers asynchronous fan-out notifications, supports standard calendar interoperability, and maintains security and privacy controls. The main area for future improvement is RSVP tail latency under sustained write load.

The remainder of this page is intentionally left blank.

7 Reflection

Building TicketTailor left several practical lessons about keeping a modular project clean, managing data across a cache and a database, and why keeping the notification worker away from the database was worth the trade-offs.

7.1 What Was Learned

Setting clear boundaries between modules pays off. Each module was given its own database tables, and no module was allowed to directly access another module's data. Import Linter checked this in CI. At first this felt like extra work – even a simple check like "does this club exist?" required a shared callback instead of a direct database join (ADR-0015). But in the end the discipline proved its worth. Each module could be understood on its own, and any module could be split into its own service if needed. This same choice would be made again.

The order of cache and database updates matters. The RSVP counter lives in Redis but PostgreSQL is the source of truth. Committing the database write *before* updating the cache is essential. If the database commit fails after the cache has been updated, a wrong count results that the nightly cleanup has to fix. Getting the order right – insert, commit, then increment – was simple but prevented a lot of headaches.

Serverless is great until hitting a connection limit. The notification worker was first planned to look up recipient details in the database. Then it became clear that sending 1000 notifications could start hundreds of Lambda functions at once, each grabbing a database connection. Moving the work of finding recipients and preparing messages to the relay (ADR-0016) kept the worker away from the database. This was one of the better decisions in the project: it avoided a problem before it happened.

7.2 What Would Be Done Differently

Clean up old push subscriptions automatically. The worker cannot reach the database, so when a push service reports a dead subscription (a 410 **Gone** error), the worker can only log it. Those dead subscriptions pile up over time. With hindsight, a way for the worker to report back would be added: it sends a cleanup event to SQS, and the relay or API picks it up and removes the stale row from the database.

Group counter updates instead of saving each RSVP one by one. The load test showed RSVP writes at 410 ms p99, above the 200 ms target (Section 6). Every RSVP gets written to PostgreSQL immediately, which is the bottleneck. With hindsight, counter changes would be kept in Redis and written to the database in batches, instead of saving each one individually. This would risk losing a small amount of data if Redis crashed, but the nightly cleanup would catch it, and writes would be much faster.

The remainder of this page is intentionally left blank.

References

- [1] Redis, “Redis race condition,” <https://redis.io/glossary/redis-race-condition/>, Redis, accessed: 05 June 2026.
- [2] C. Richardson, *Microservices Patterns*. S.l.: Manning Publications, 2025, oCLC: 1536315654.
- [3] iCalendar.org, “icalendar resources,” <https://icalendar.org/resources.html>, iCalendar.org, accessed: 05 June 2026.

A Supplementary Diagrams

This appendix contains entity-relationship and sequence diagrams that supplement the architecture description in Section 3.

A.1 Entity-Relationship Diagram

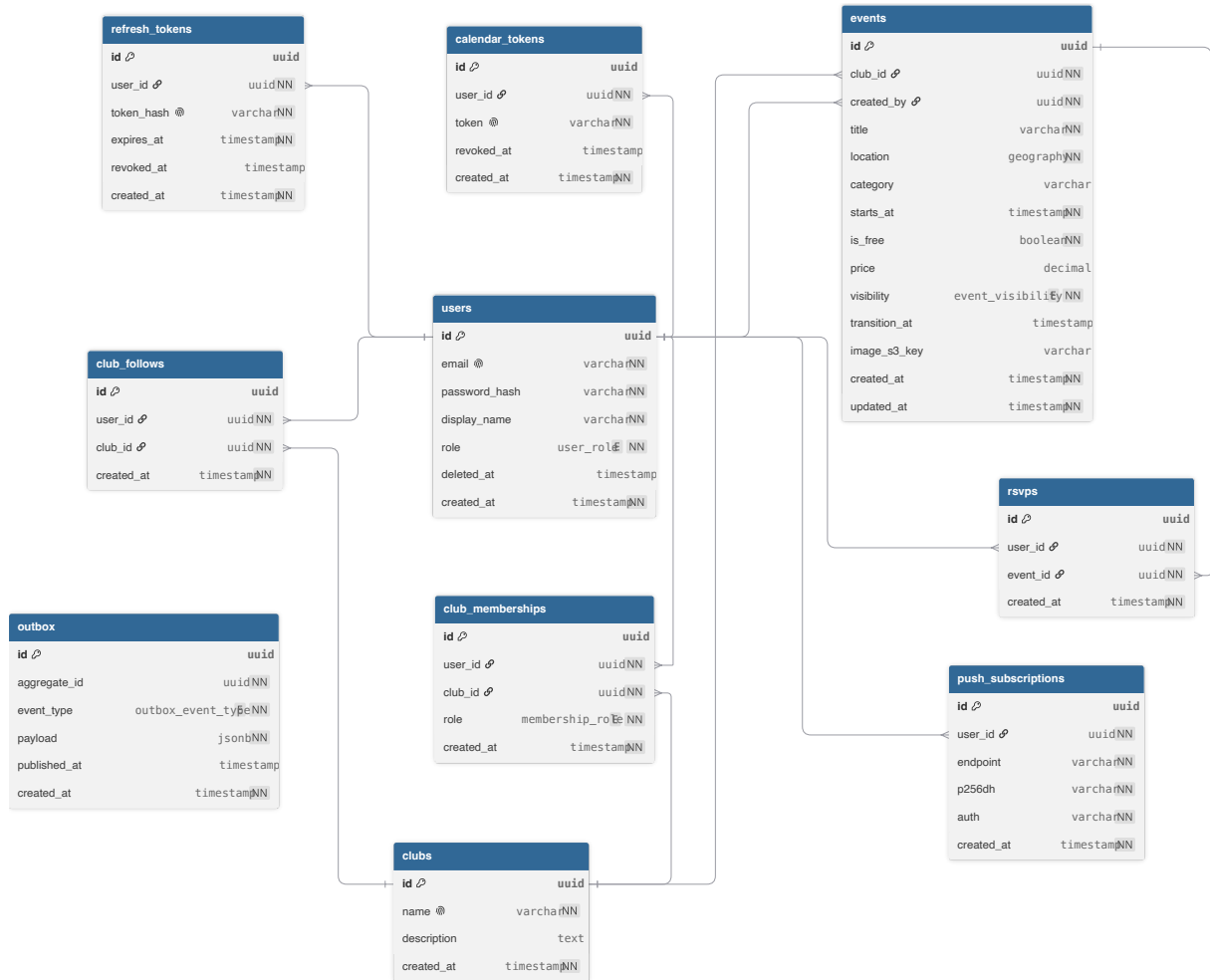


Figure 5: Database entity-relationship model showing the main tables and their relationships across bounded modules.

A.2 Sequence Diagram: RSVP Placement

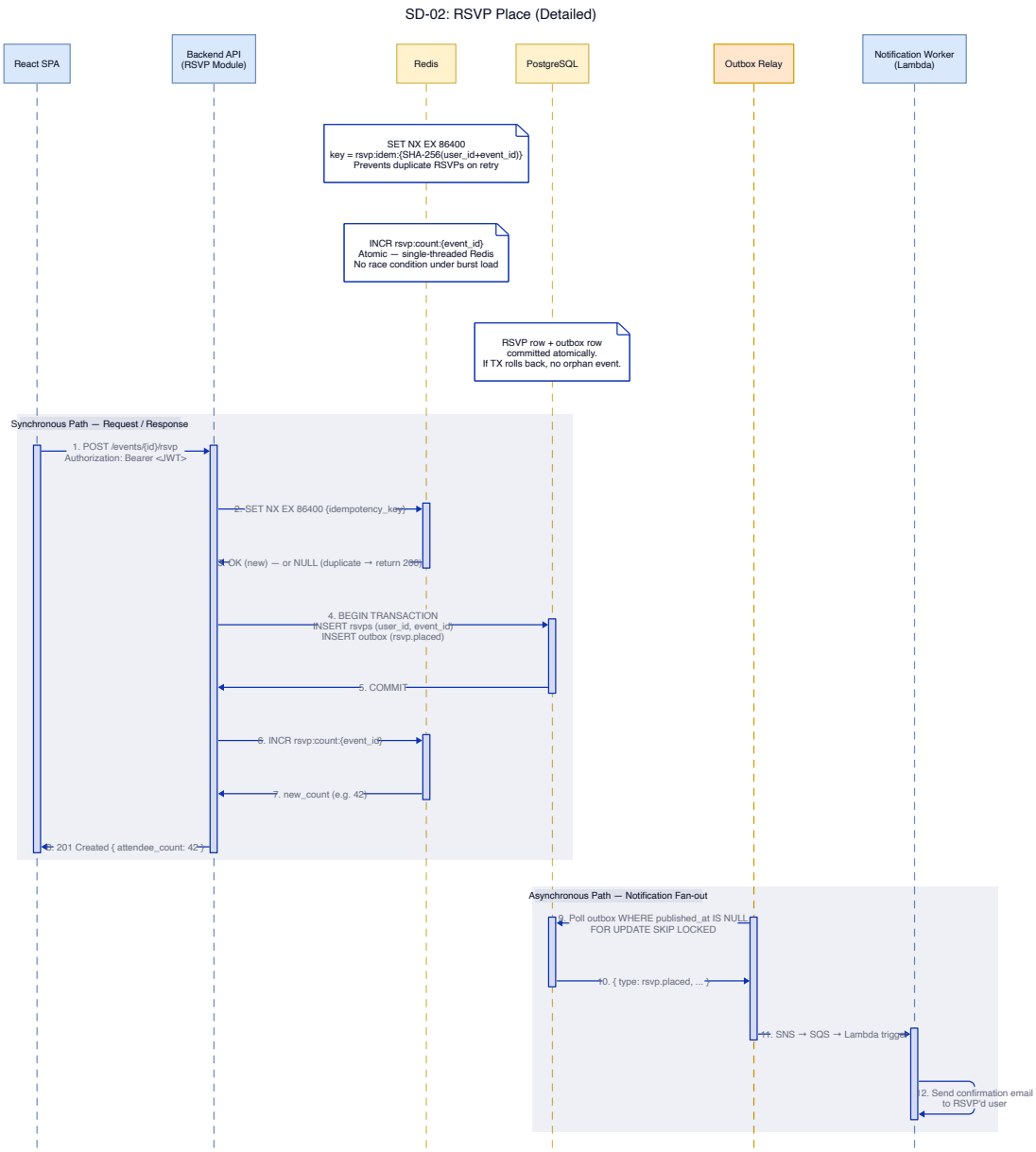


Figure 6: Sequence diagram for RSVP placement, showing the strict order of operations: event lookup, authorisation, idempotency check, counter seed, row insert, transactional outbox, commit, then cache increment (ADR-0005).

A.3 Sequence Diagram: Authentication Flow

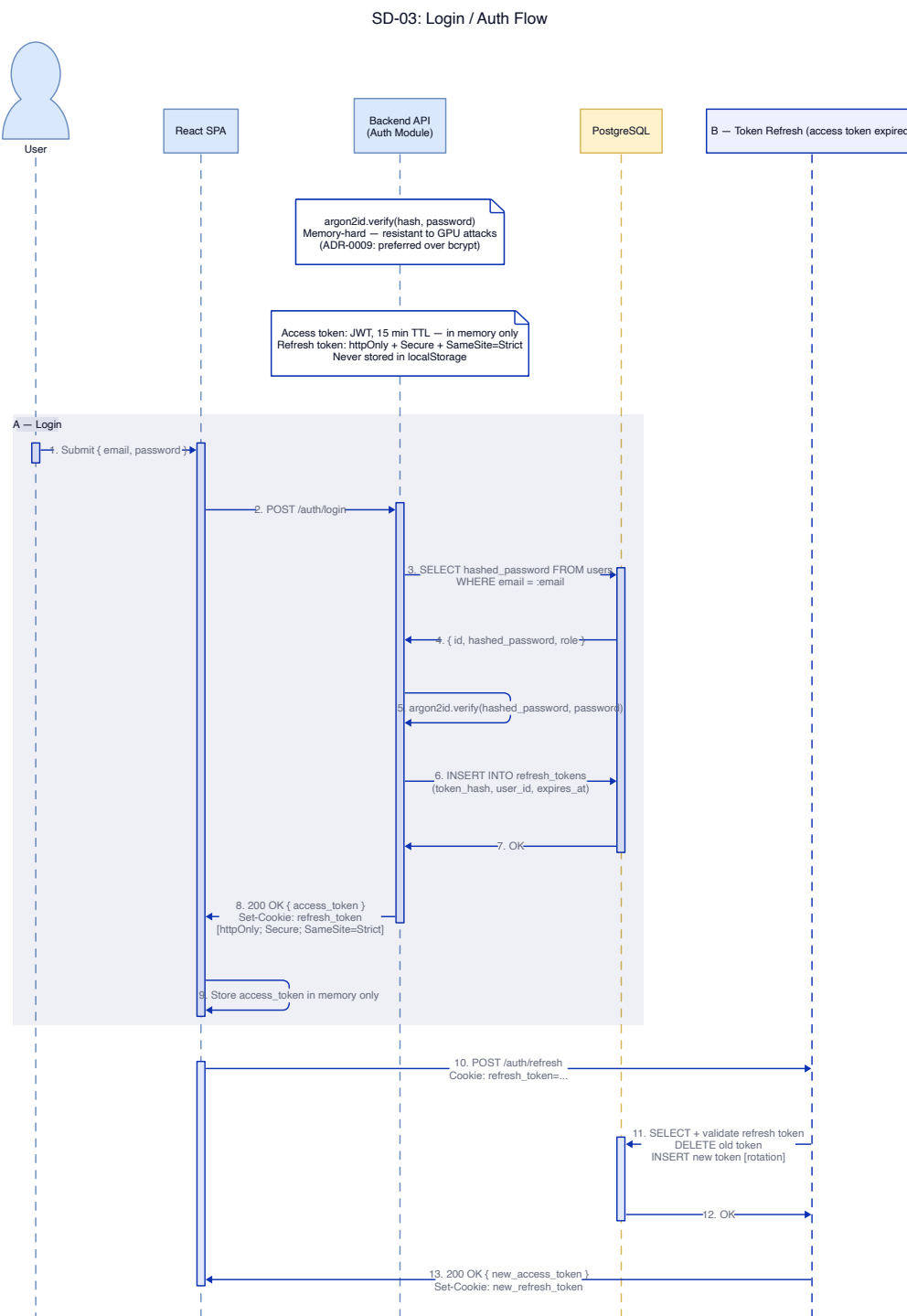


Figure 7: Authentication sequence covering registration, login with short-lived access token and rotating refresh token, and token refresh (ADR-0014).

A.4 Sequence Diagram: Visibility Transition

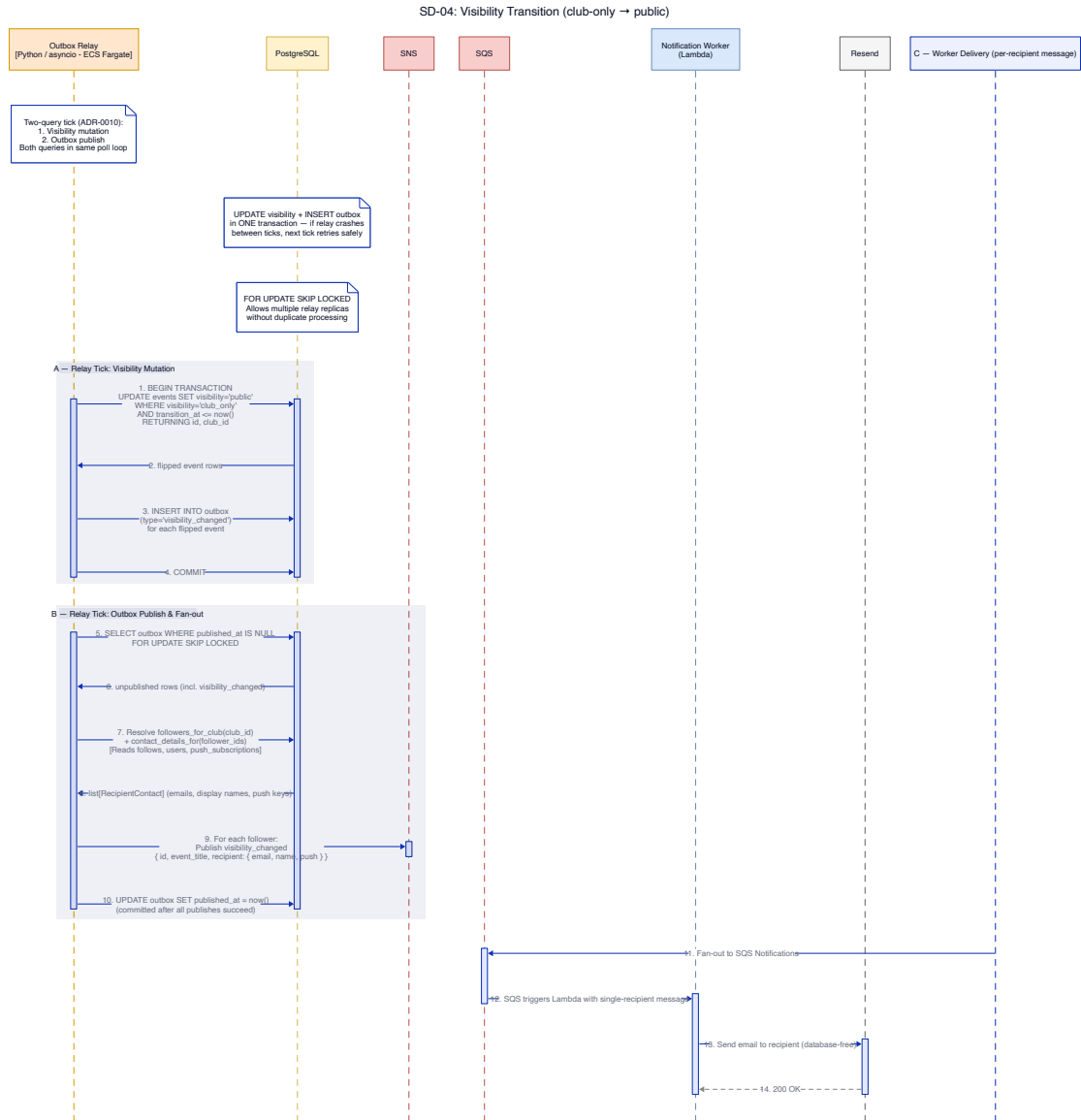


Figure 8: Visibility transition flow: the relay flips club-only events to public at their scheduled time, then publishes a `visibility_changed` event through the outbox pipeline (ADR-0010).

A.5 Sequence Diagram: Browse Events

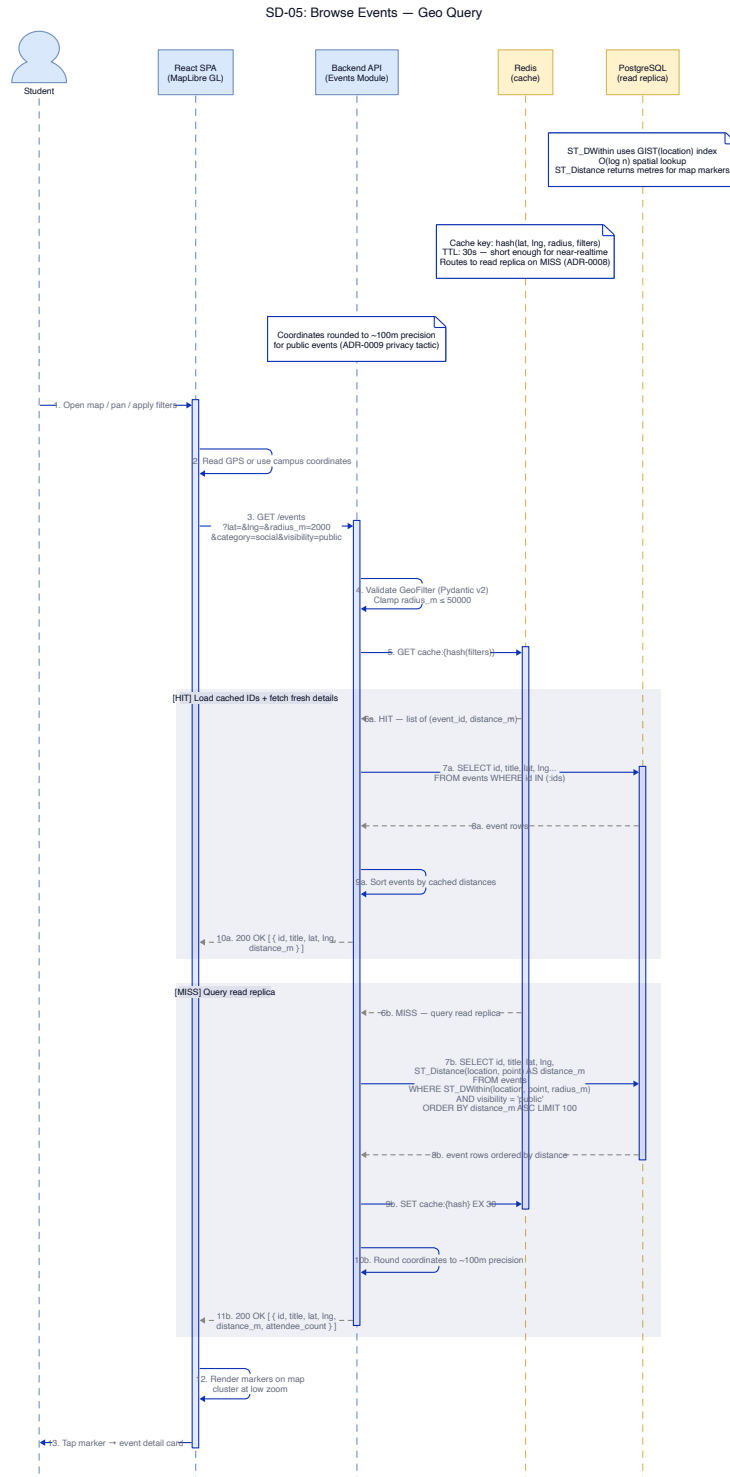


Figure 9: Event browsing flow: a student browses public events on the map with geo-radius and category filters, with queries served from the read replica.

A.6 Sequence Diagram: Calendar Export

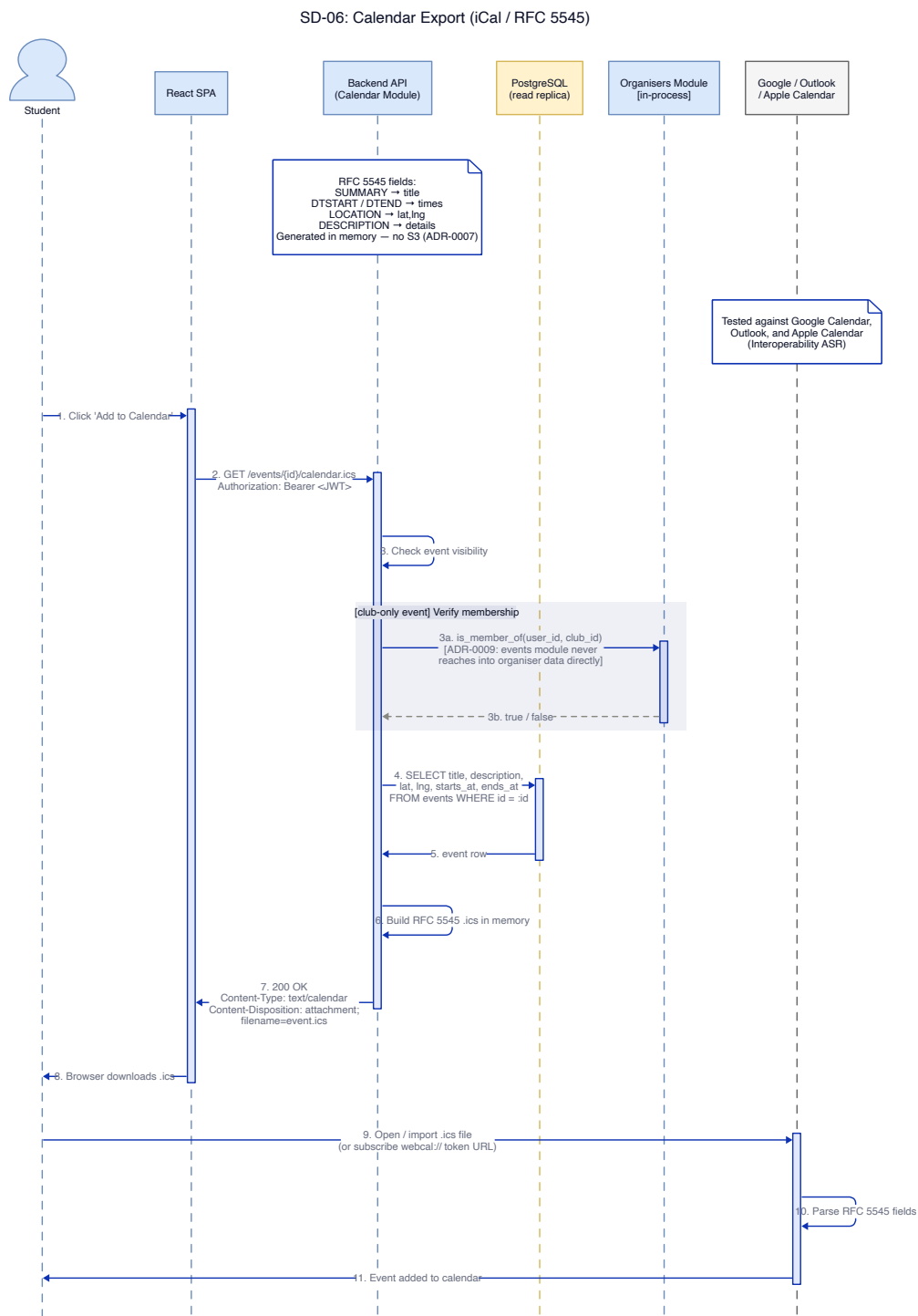


Figure 10: Calendar export flow: a user generates an RFC 5545 iCalendar feed and imports it into an external calendar client.

Declaration of AI Usage

- I, Jordan Lazo Cahua, acknowledge the use of AI tools to support the writing of the Trade-off and Critique sections in this report. Gemini was utilized to generate an initial draft, while DeepL and Grammarly were used for translation and grammar refinement. All final decisions, content, and contributions remain entirely my own.
- I, Irene Ge, acknowledge the use of ChatGPT to support the writing and review of the Architecture Options and Evaluation sections in this report. ChatGPT was used for structuring, wording refinement, evidence checking, and LaTeX formatting assistance. All final decisions, content, and contributions remain my own.